

QueryQuest – Assignment 2

Aditi
240048

Part I

Q1. The reason behind the failure of this query is that the NULL is unknown and we can not use standard comparison operators like = or !=, kinda things. The correct query is

```
SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;
```

NULL is excluded from most aggregate computations and comparison operations because Aggregate functions are designed to look at the data that actually exists. If aggregate functions included NULL, any mathematical calculation involving an unknown value will result in unknown, ruining the calculation. In SQL, a condition can evaluate to True, False, or Unknown. As a NULL value is an unknown quantity, the database cannot determine if it is equal to, greater than, or less than another value.

Q2. Yes, the student is correct. The DISTINCT keyword in this query is completely redundant. Because the GROUP BY department clause guarantees that each department will appear only once in the final output and applying DISTINCT forces the database to perform an unnecessary deduplication step on a result set that is already unique.

Q3. The difference in row count tells that there are customers in the database who have never placed an order.

The scenario that would cause the LEFT JOIN to return more rows is the presence of new or inactive customers. Consider, if a restaurant has 10 registered customers in which 2 of them have not made any purchase yet (let them be new users). The INNER JOIN between Customer and Order will only return rows for the 8 customers who have active orders BUT the LEFT JOIN will return all 10 customers. This results in the LEFT JOIN returning 2 more rows than the INNER JOIN.

Q4. This query is invalid because grouping collapses multiple rows into a single summary row per department and standard SQL strictly prohibits selecting granular, un-grouped columns (employee_name) as it refuses to make an arbitrary guess on which individual's name to display.

If SQL conceptually allowed this query to run, it would mean the database engine would have to choose a single value for employee_name out of multiple distinct possibilities within a department.

The correct version is

```
SELECT department, AVG(salary)
```

```
FROM Employee
GROUP BY department;
```

Q5. The error in this query is the use of the aggregate function AVG(salary) inside the WHERE clause.

The logical order in which the database engine actually executes a query FROM → WHERE → GROUP BY → SELECT which is different from the order in which it is written. Because the WHERE clause runs long before the AVG() function is even computed, the database engine throws a syntax error.

The correct query is

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. (a) A **non-correlated subquery** is appropriate here

because the overall average price is a single, static value that only needs to be calculated once for the entire table. The inner query can execute independently of the outer query, allowing the database to compute the average first and use that fixed result to filter all products.

(b) A **correlated subquery** is appropriate here

because the average salary changes depending on which employee's row is currently being evaluated. The inner query must explicitly reference the department of the current row from the outer query, meaning the average must be re-calculated dynamically for each specific department.

Part II

(a) Used a LEFT JOIN to return all customers from the Customer table regardless of their order history, combining it with a WHERE o.order_id IS NULL filter to isolate only those records where no matching order exists. I used this because a LEFT JOIN provides a clear, dialect-neutral way to identify unmatched records.

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN Order o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

(b) Used an inner subquery to first calculate the discrete total value of every individual order before grouping them by restaurant in the outer query to compute their averages. The HAVING clause is required here for filtering out restaurants with 5 or fewer orders which

relies on a count-based aggregate condition (COUNT(DISTINCT o.order_id)), which cannot be processed inside a WHERE clause.

```
SELECT o.restaurant_id, AVG(order_totals.total_price) AS avg_order_value
FROM Order o
JOIN (
    SELECT order_id, SUM(price * quantity) AS total_price
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY order_id
) AS order_totals ON o.order_id = order_totals.order_id
GROUP BY o.restaurant_id
HAVING COUNT(DISTINCT o.order_id) > 5;
```

(c) Used a non-correlated subquery to calculate the platform-wide average metric. A correlated subquery is completely unnecessary here because the baseline comparison value, the overall average of all restaurants, is a static figure that remains identical for every row evaluated by the outer query.

```
WITH RestaurantAverages AS (
    SELECT o.restaurant_id, AVG(order_totals.total_price) AS avg_order_value
    FROM Order o
    JOIN (
        SELECT order_id, SUM(price * quantity) AS total_price
        FROM OrderItem oi
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY order_id
    ) AS order_totals ON o.order_id = order_totals.order_id
    GROUP BY o.restaurant_id
    HAVING COUNT(DISTINCT o.order_id) > 5
)
SELECT restaurant_id, avg_order_value
FROM RestaurantAverages
WHERE avg_order_value > (SELECT AVG(avg_order_value) FROM RestaurantAverages);
```

(d) SELECT item_totals.restaurant_id, item_totals.item_id, item_totals.item_name,
item_totals.total_qty
FROM (
 SELECT mi.restaurant_id, mi.item_id, mi.item_name, SUM(oi.quantity) AS total_qty,
 RANK() OVER (PARTITION BY mi.restaurant_id ORDER BY SUM(oi.quantity) DESC) AS
item_rank
 FROM MenuItem mi
 JOIN OrderItem oi ON mi.item_id = oi.item_id
 GROUP BY mi.restaurant_id, mi.item_id, mi.item_name

) AS item_totals

WHERE item_totals.item_rank = 1;

The main limitation of a basic GROUP BY paired with a MAX() function is handling ties. If a restaurant has two different menu items that are tied for the highest number of orders, a basic MAX() approach forces you to match back to the table arbitrarily and will often display only one of them.

Part III

Q1. The functional dependencies implied here are:

student_id → student_name

(primary composite key)

course_id → course_name, instructor_id

(primary composite key)

instructor_id → instructor_name, instructor_office

(student_id, course_id) → grade, enrollment_date

Q2. Yes, this table is in 1NF.

Because in this scenario, every cell in a given row holds exactly one piece of information (one student ID, one course name, one grade), satisfying the fundamental structural criteria of 1NF.

Q3. The partial dependencies violating 2NF are:

student_id → student_name : The student's name depends only on the student_id, completely ignoring the course_id.

course_id → course_name, instructor_id : The course details and instructor information depend entirely on the course_id, completely ignoring which student is enrolled.

It is a problem because these attributes do not depend on the entire primary key, they force to repeat independent data every single time a new enrollment occurs. All the three type of anomalies can occur.

INSERTION ANOMALY

Suppose the university creates a brandnew course, "Theories of Spider-Man (SM042)", taught by Dr. Strange. Because the primary key of this table requires a student_id, the university cannot insert this new course into the database until a student actually enrolls in it. Attempting to insert the course with a blank student_id will fail because primary keys can never contain NULL values.

DELETION ANOMALY

Imagine there is a specialized course called "Theory of Miles Morales (MM302)" that only has one student enrolled. If that student decides to drop the course and we have to delete their row to clear their enrollment record, but we simultaneously wipe out all record of the course's existence, its name, its instructor's ID, and where that instructor's office is located. The entire course data vanishes from the university system as a side effect of a student

dropping out.

UPDATE ANOMALY

If 500 students are enrolled in "Theories of Spider-Man (SM042)" and the course name changes to "The Physics of the Spider-Verse", the database must locate and update all 500 individual student rows. If the system crashes midway or misses even a single row, the database falls into an inconsistent state where the exact same course ID maps to two different names depending on which student's record we query.

Q4. The transitive dependency violating 3NF is

$\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$

This transitive dependency detaches the instructor's personal data from the course itself, causing an independent entity anomaly. This makes it impossible to insert a new instructor who isn't currently teaching a class, and results in the accidental deletion of an instructor's data if the only course they teach is removed.

Q5. STUDENT TABLE

Student(<u>student_id</u>, student_name)

Separated out due to the functional dependency. This eliminates a partial dependency on the composite primary key, ensuring a student's name is recorded only once.

INSTRUCTOR TABLE

Instructor(<u>instructor_id</u>, instructor_name, instructor_office)

Separated out due to the functional dependency. This eliminates a transitive dependency, allowing instructor details to exist independently of any course assignments.

COURSE TABLE

Course(<u>course_id</u>, course_name, instructor_id)

Separated out due to the functional dependency course. This eliminates a partial dependency on the composite key, tracking which instructor teaches which course without repeating it for every enrolled student.

ENROLLMENT TABLE

Enrollment(<u>student_id</u>, <u>course_id</u>, grade, enrollment_date)

This relation represents the core many-to-many relationship, justified by the functional dependency. Since all remaining attributes depend fully on the entire composite key, this table is now in 3NF.

Part IV

- In Part II(a), I chose a LEFT JOIN combined with a WHERE NULL filter over a NOT IN subquery to isolate customers who have never ordered. While both approaches yield identical results under ideal circumstances, a NOT IN subquery can silently fail and

return an empty result set if any row in the subquery's result evaluated to NULL. The LEFT JOIN format provides reliable, dialect-neutral handling of missing data, ensuring correct execution regardless of database anomalies, while remaining highly readable to other developers.

- While normalizing the Enrollment table to 3NF in Part III eliminates structural anomalies and data redundancy, it fundamentally trades off against query convenience. In the unnormalized "mega-table," a manager can fetch a complete breakdown of students, courses, and instructor office rooms using a single, rapid SELECT * query. Once split into 3NF, the database must dynamically piece that ecosystem back together using three separate, resource-intensive JOIN operations. A real-world production system might intentionally retain some denormalization to optimize read-heavy workloads, eliminate expensive join operations, and significantly reduce query latency.